

Universidad Torcuato Di Tella

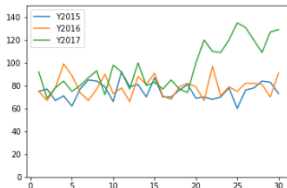
Métodos Analíticos y Programación

Prof. Hernán Czemerinski

PANDAS: Series + Introducción a DataFrames

¿Qué es Pandas?

	SEP	Y2015	Y2016	Y2017
0	1	75	75	92
1	2	77	67	69
2	3	67	78	78
3	4	71	99	84
4	5	62	89	75



Pandas es una librería de Python diseñada para el manejo y análisis de datos. Permite (entre otras cosas):

- ▶ Leer y escribir datos (CSV, Excel, SQL, JSON, etc.)
- ▶ Limpiar, transformar y preparar datos
- ▶ Hacer análisis estadístico
- ▶ Filtrar, agrupar y combinar datos
- ▶ Trabajar con series temporales

Clases Provistas por Pandas

Pandas provee distintas clases para trabajar con datos.

- ▶ **Series**: estructura de una dimensión
- ▶ **DataFrame**: estructura de dos dimensiones (tablas)
- ▶ **Panel**: estructura de tres dimensiones (cubos)
(no veremos en la materia)

Las series son similares a las [listas](#) de Python, pero...

- ▶ Todos sus elementos deben ser del mismo tipo
- ▶ No se puede cambiar su dimensión (sí su contenido)

```
import pandas as pd

s = pd.Series(['Matemática', 'Historia', 'Economía', 'Programación'])
print(s[1]) # Imprime 'Historia'
print(s)
''' Imprime:
      0    Matemática
      1    Historia
      2    Economía
      3    Programación
      dtype: object
'''
s[1] = 'Inglés'
print(s[1]) # Imprime 'Inglés'
```

También pueden ser similares a los **diccionarios** de Python.

```
import pandas as pd

s = pd.Series({'Matemática': 6.0, 'Economía': 4.5, 'Programación': 8.5})
print(s['Economía']) # Imprime 4.5
print(s)
''' Imprime:
      Matemática      6.0
      Economía      4.5
      Programación    8.5
      dtype: float64
'''
s['Economía'] = 7.3
print(s['Economía']) # Imprime 7.3
```

Otra forma de crear lo mismo.

```
s = pd.Series([6.0, 4.5, 8.5], index = ['Matemática', 'Economía',
                                       'Programación'])
```

Atributos y métodos de la clase Series:

- ▶ `s.size`: cantidad de elementos de la serie `s`.
- ▶ `s.index`: lista con los nombres de los índices de la serie `s`.

Acceso por posición

- ▶ `s[i]`: devuelve el *i*-ésimo elemento de la serie `s`.
- ▶ `s[posiciones]`: devuelve una nueva serie con los elementos ubicados en las posiciones indicadas en la lista `posiciones`.
- ▶ `s[i:j]`: devuelve una nueva serie con los elementos ubicados en las posiciones determinadas por el *slicing* `[i:j]`.

```
import pandas as pd

s1 = pd.Series({'Matemática': 6.0, 'Economía': 4.5, 'Programación': 8.5})
s2 = s1[1:]
print(s2)
''' Imprime:
           Economía      4.5
        Programación      8.5
           dtype: float64
'''
```

Atributos y métodos de la clase Series:

- ▶ `s.size`: cantidad de elementos de la serie `s`.
- ▶ `s.index`: lista con los nombres de los índices de la serie `s`.

Acceso por etiqueta

- ▶ `s[nombre]`: devuelve el elemento cuya etiqueta de índice es `nombre`.
- ▶ `s[nombres]`: devuelve una nueva serie que contiene los elementos cuyas etiquetas de índice coinciden con los nombres indicados en la lista `nombres`.

```
import pandas as pd

s1 = pd.Series({'Matemática': 6.0, 'Economía': 4.5, 'Programación': 8.5})
s2 = s1[['Matemática', 'Programación']]
print(s2)
''' Imprime:
      Matemática      6.0
      Programación    8.5
      dtype: float64 '''
```

Métodos para ordenar Series:

- `s.sort_values(ascending=[bool])`: Devuelve una nueva Series que resulta de ordenar los valores la serie `s`. Si argumento del parámetro `ascending` es `True` el orden es creciente y si es `False` decreciente.

```
s1 = pd.Series({'Economía': 6.0, 'Matemáticas': 4.5, 'Programación': 8.5})
s2 = s1.sort_values()
print(s2)
''' Imprime:
    Matemáticas      4.5
    Economía        6.0
    Programación     8.5
    dtype: float64 '''
```

- `s.sort_index(ascending=[bool])`: Devuelve una nueva Series que resulta de ordenar `s` por el índice. Si el argumento del parámetro `ascending` es `True`, el orden es creciente; si es `False`, el orden es decreciente.

```
s1 = pd.Series({'Economía': 6.0, 'Matemáticas': 4.5, 'Programación': 8.5})
s2 = s1.sort_index(ascending=False)
print(s2)
''' Imprime:
    Programación      8.5
    Matemáticas       4.5
    Economía          6.0
    dtype: float64 '''
```


Otros métodos de Series:

- ▶ `s.unique()`: devuelve los valores únicos de una Series, en el orden en que aparecen por primera vez.
- ▶ `s.nunique()`: devuelve un entero con la cantidad de valores distintos en una Series.
- ▶ `s.sum()`: devuelve la suma de los valores de la serie `s`, si son numéricos, o su concatenación si son cadenas de texto.
- ▶ `s.min()`: devuelve el valor mínimo de la serie `s`.
- ▶ `s.max()`: devuelve el valor máximo de la serie `s`.
- ▶ `s.mean()`: devuelve el promedio de los valores de la serie `s`, si son numéricos.

```
import pandas as pd

s = pd.Series([1,2,2,3,1])
print(s.unique())      # Imprime [1 2 3]
print(s.nunique())     # Imprime 3
print(s.min())         # Imprime 1
print(s.max())         # Imprime 3
print(s.mean())        # Imprime 1.8
```

Ejercicio 1. Se registraron las temperaturas promedio (en °C) de algunas ciudades durante la última semana:

Ciudad	Temperatura
Buenos Aires	22
Córdoba	18
Mendoza	25
Rosario	20
Salta	26

1. Crear una Series de pandas llamada `temps`, donde el índice sea el nombre de la ciudad y los valores sean las temperaturas.
2. Escribir un programa que calcule y muestre:
 - ▶ La cantidad de ciudades con temperatura registrada.
 - ▶ La suma de las temperaturas registradas.
 - ▶ La temperatura mínima.
 - ▶ La temperatura máxima.
 - ▶ La temperatura promedio.
 - ▶ Las ciudades ordenadas por temperatura (de menor a mayor).

DataFrame

Un `DataFrame` es una clase para representar datos estructurados en forma de tabla:

	Nombre	Edad	Carrera	Correo
0	María	18	Economía	maria@gmail.com
1	Luis	22	Medicina	luis@yahoo.es
2	Carmen	20	Arquitectura	carmen@gmail.com
3	Antonio	21	Economía	antonio@gmail.com

- ▶ Cada columna es un objeto `Series`, con datos del mismo tipo.
- ▶ Cada fila es un registro, que puede combinar distintos tipos de datos.
- ▶ Tiene dos índices (filas y columnas) y permite acceder a los elementos por sus nombres.

DataFrame

	Nombre	Edad	Carrera	Correo
0	María	18	Economía	maria@gmail.com
1	Luis	22	Medicina	luis@yahoo.es
2	Carmen	20	Arquitectura	carmen@gmail.com
3	Antonio	21	Economía	antonio@gmail.com

Un **DataFrame** se puede crear a partir de un diccionario:

```
datos = {
    "Nombre":  ["María", "Luis", "Carmen", "Antonio"],
    "Edad":    [18, 22, 20, 21],
    "Carrera": ["Economía", "Medicina", "Arquitectura", "Economía"],
    "Correo":  ["maria@gmail.com", "luis@yahoo.es", "carmen@gmail.com",
                "antonio@gmail.com"]
}

df = pd.DataFrame(datos)
print(df)
''' Imprime:
      Nombre  Edad  Carrera  Correo
0     María   18   Economía  maria@gmail.com
1      Luis   22   Medicina  luis@yahoo.es
2    Carmen   20  Arquitectura  carmen@gmail.com
3   Antonio   21   Economía  antonio@gmail.com '''
```

DataFrame

Un `DataFrame` se puede crear a partir de un diccionario.

```
df = DataFrame(data=diccionario, index=..., columns=...)
```

- ▶ `data`:

- ▶ Claves = nombres de columnas
- ▶ Valores = listas con datos

- ▶ Opcionales:

- ▶ `index`: nombres de filas (default: 0,1,2,...)
- ▶ `columns`: nombres de columnas (default: claves de data)

También pueden crearse a partir de listas de listas, una lista de diccionarios o arrays de la librería `numpy`, pero no trabajaremos con eso.

DataFrame

Nosotros crearemos **DataFrames** a partir de archivos CSV mediante la función `read_csv(...)` provisto por pandas.

```
import pandas as pd

df = pd.read_csv('colesterol.csv')
print(df)

''' Imprime:
      nombre  edad  sexo  peso  altura  colesterol
0  Martín González   18    H    85    1.79         182
1   Lucía Fernández   32    M    65    1.73         232
2    Sofía López     35    M    65    1.70         200
3   María Gómez     46    M    51    1.58         148
4    Juan Ruiz      68    H    66    1.74         249
5  Diego Fernández   51    H    62    1.72         276
6   Pedro Sánchez   35    H    90    1.94         241
7 Santiago Rodríguez  46    H    75    1.85         280
8  Macarena Álvarez   53    M    55    1.62         262
9  José De la Fuente   58    H    78    1.87         198
10 Miguel Cuadrado   27    H   109    1.98         210
11  Carolina Rubio    20    M    61    1.77         194
'''
```

Dado un DataFrame `df`, `df.to_csv(archivo_csv, index=[bool])` guarda su contenido en `archivo_csv`. Con `index=True` se incluyen los nombres de las filas; con `False`, no.

DataFrame

Algunos métodos de DataFrame:

- ▶ `df.head(n)`: devuelve un nuevo DataFrame con los primeros `n` registros de `df` (por defecto, 5).
- ▶ `df.tail(n)`: devuelve un nuevo DataFrame con los últimos `n` registros de `df` (por defecto, 5).
- ▶ `len(df)`: devuelve la cantidad de registros de `df`.

```
import pandas as pd

df = pd.read_csv('colesterol.csv')
primeros_df = df.head()
print(primeros_df)

''' Imprime:
      nombre  edad  sexo  peso  altura  colesterol
0  Martín González   18    H   85    1.79         182
1  Lucía Fernández   32    M   65    1.73         232
2      Sofía López   35    M   65    1.70         200
3   María Gómez   46    M   51    1.58         148
4      Juan Ruiz   68    H   66    1.74         249
'''

print(len(primeros_df))      # Imprime 5
```

DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **posiciones**:

- ▶ `df.iloc[i, j]`: devuelve el elemento en la fila `i` y columna `j`.
- ▶ `df.iloc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas `filas` y `columnas`.
- ▶ `df.iloc[i]`: devuelve un objeto `Series` con los elementos de la fila `i`.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H    85    1.79         182
1  Lucía Fernández   32   M    65    1.73         232
2    Sofía López    35   M    65    1.70         200
3   María Gómez    46   M    51    1.58         148
4    Juan Ruiz    68   H    66    1.74         249
'''

print(primeros_df.iloc[1,3])    # imprime 65
```


DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **posiciones**:

- ▶ `df.iloc[i, j]`: devuelve el elemento en la fila `i` y columna `j`.
- ▶ `df.iloc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas `filas` y `columnas`.
- ▶ `df.iloc[i]`: devuelve un objeto `Series` con los elementos de la fila `i`.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H    85    1.79         182
1  Lucía Fernández   32   M    65    1.73         232
2    Sofía López    35   M    65    1.70         200
3   María Gómez    46   M    51    1.58         148
4    Juan Ruiz    68   H    66    1.74         249
'''

print(primeros_df.iloc[[0,2],[0,3,4]])
''' Imprime:
   nombre  peso  altura
0  Martín González    85    1.79
2    Sofía López    65    1.70
'''
```

DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **posiciones**:

- ▶ `df.iloc[i, j]`: devuelve el elemento en la fila `i` y columna `j`.
- ▶ `df.iloc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas filas y columnas.
- ▶ `df.iloc[i]`: devuelve un objeto Series con los elementos de la fila `i`.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González  18   H   85    1.79         182
1  Lucía Fernández  32   M   65    1.73         232
2    Sofía López    35   M   65    1.70         200
3  María Gómez    46   M   51    1.58         148
4    Juan Ruiz    68   H   66    1.74         249
'''

print(primeros_df.iloc[2])
''' Imprime:
nombre    Sofía López
edad              35
sexo              M
peso              65
altura           1.7
colesterol       200
Name: 2, dtype: object
'''
```

DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **nombres**:

- ▶ `df.loc[fila, columna]`: devuelve el elemento en la fila con nombre fila y la columna con nombre columna.
- ▶ `df.loc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas filas y columnas.
- ▶ `df[columna]`: devuelve una nueva Series con los elementos de columna.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
      nombre  edad  sexo  peso  altura  colesterol
0  Martín González   18    H    85    1.79         182
1   Lucía Fernández   32    M    65    1.73         232
2      Sofía López    35    M    65    1.70         200
3   María Gómez     46    M    51    1.58         148
4     Juan Ruiz     68    H    66    1.74         249
'''

print(primeros_df.loc[2, 'colesterol'])    # imprime 200
```

DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **nombres**:

- ▶ `df.loc[fila, columna]`: devuelve el elemento en la fila con nombre `fila` y la columna con nombre `columna`.
- ▶ `df.loc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas `filas` y `columnas`.
- ▶ `df[columna]`: devuelve una nueva Series con los elementos de columna.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H    85    1.79         182
1  Lucía Fernández   32   M    65    1.73         232
2    Sofía López    35   M    65    1.70         200
3   María Gómez    46   M    51    1.58         148
4    Juan Ruiz     68   H    66    1.74         249
'''

print(primeros_df.loc[[2,4], ['colesterol', 'peso']])
''' Imprime:
   colesterol  peso
2          200    65
4          249    66
'''
```

DataFrame

El acceso a los datos de un DataFrame puede hacerse por posiciones o por nombres de filas y columnas.

Acceso mediante **nombres**:

- ▶ `df.loc[fila, columna]`: devuelve el elemento en la fila con nombre `fila` y la columna con nombre `columna`.
- ▶ `df.loc[filas, columnas]`: devuelve un nuevo DataFrame con los datos indicados en las listas `filas` y `columnas`.
- ▶ `df[columna]`: devuelve una nueva Series con los elementos de `columna`.

```
primeros_df = df.head()
print(primeros_df)
''' Imprime:
   nombre  edad  sexo  peso  altura  colesterol
0  Martín González   18    H   85    1.79         182
1  Lucía Fernández   32    M   65    1.73         232
2    Sofía López    35    M   65    1.70         200
3   María Gómez    46    M   51    1.58         148
4    Juan Ruiz     68    H   66    1.74         249
'''

print(primeros_df['altura'])
''' Imprime:
0    1.79
1    1.73
2    1.70
3    1.58
4    1.74
Name: altura, dtype: float64
'''
```

Métodos para ordenar un DataFrame:

- `df.sort_values(columna, ascending=booleano)`: devuelve un nuevo DataFrame ordenado por la columna indicada. Si `ascending=True` el orden es creciente, si es `False` es decreciente.

```
print(df.sort_values('colesterol'))
''' Imprime:
      nombre  edad sexo  peso  altura  colesterol
3   María Gómez   46   M   51   1.58         148
0   Martín González  18   H   85   1.79         182
11  Carolina Rubio   20   M   61   1.77         194
9   José De la Fuente  58   H   78   1.87         198
... '''
```

- `df.sort_values(by=[col_1, col_2, ...], ascending=[b_1, b_2, ...])` devuelve un nuevo DataFrame ordenando las filas de `df` según las columnas listadas en `by`. Cada valor de la lista `ascending` indica si el orden en la columna correspondiente es creciente (`True`) o decreciente (`False`).

```
df_ordenado = df.sort_values(by=["sexo", "colesterol"],
                             ascending=[True, False])
print(df_ordenado)
''' Imprime:
      nombre  edad sexo  peso  altura  colesterol
9   José De la Fuente  58   H   78   1.87         198
0   Martín González  18   H   85   1.79         182
11  Carolina Rubio   20   M   61   1.77         194
3   María Gómez   46   M   51   1.58         148
... '''
```

Ejercicio 2. Se cuenta con un archivo CSV con la siguiente estructura:

```
nombre,edad,carrera,promedio,correo  
Ana Pérez,21,Ingeniería,8.5,ana.perez@gmail.com  
Juan López,19,Medicina,9.2,juan.lopez@yahoo.com  
...
```

Se pide:

1. Crear un DataFrame `df_estudiantes` a partir del archivo `estudiantes.csv`.
2. Mostrar las primeras filas del DataFrame `df_estudiantes`.
3. Crear y mostrar un DataFrame con las columnas `nombre` y `promedio` de las filas 2 y 4.
4. Crear un nuevo DataFrame llamado `df_promedios_dec` a partir de `df_estudiantes`, ordenado por `promedio` en orden descendente y mostrar sus primeras filas.
5. Mostrar el registro del estudiante con el promedio más alto.
6. Guardar `df_promedios_dec` en el archivo CSV `estudiantes_por_promedio.csv`, sin los nombres de fila.

Repaso de la clase de hoy

Referencia: [Documentación de pandas \(link\)](#)

- ▶ Series
- ▶ DataFrame

Con lo visto, ya pueden resolver hasta el ejercicio 1.3 de la guía de Métodos Analíticos.